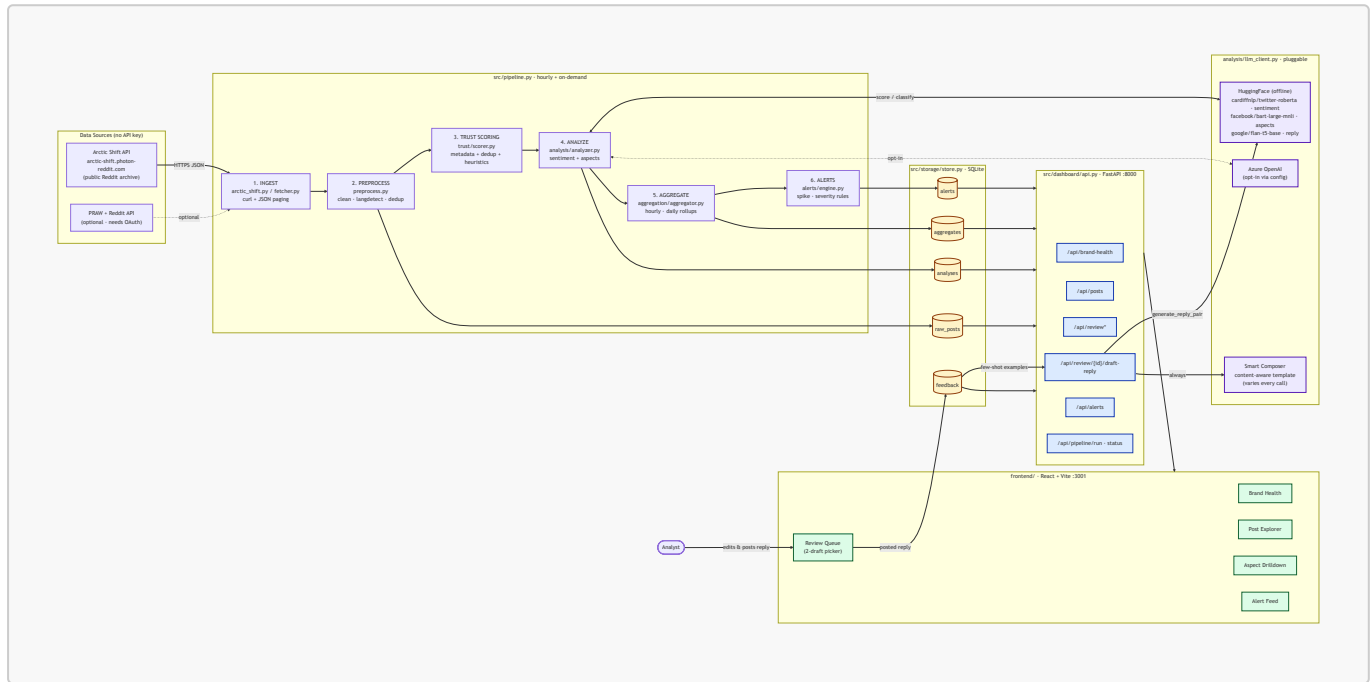


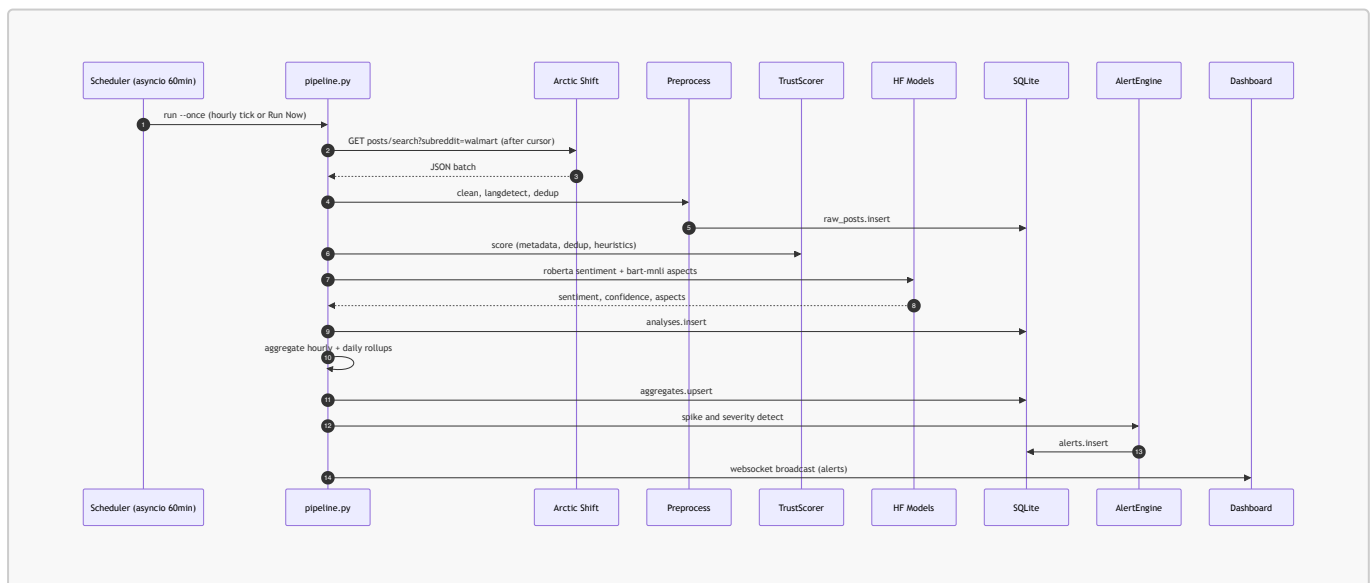
Retail Sentiment Intelligence — Architecture, Pipeline & Tools

Living reference for how data flows through the system, which tools we use at each layer, and how the LLM "learning loop" works for analyst replies.

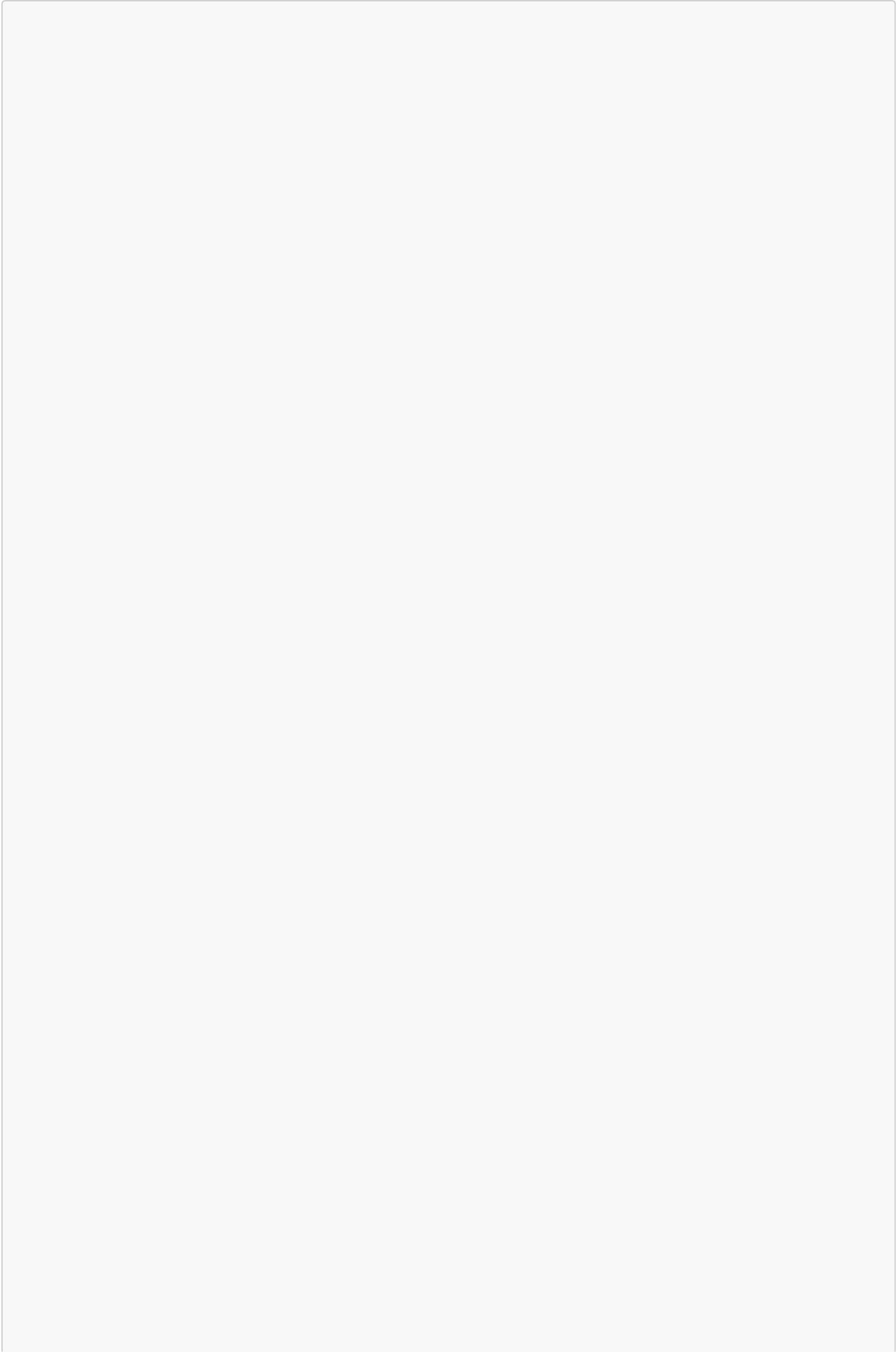
1. End-to-end data flow

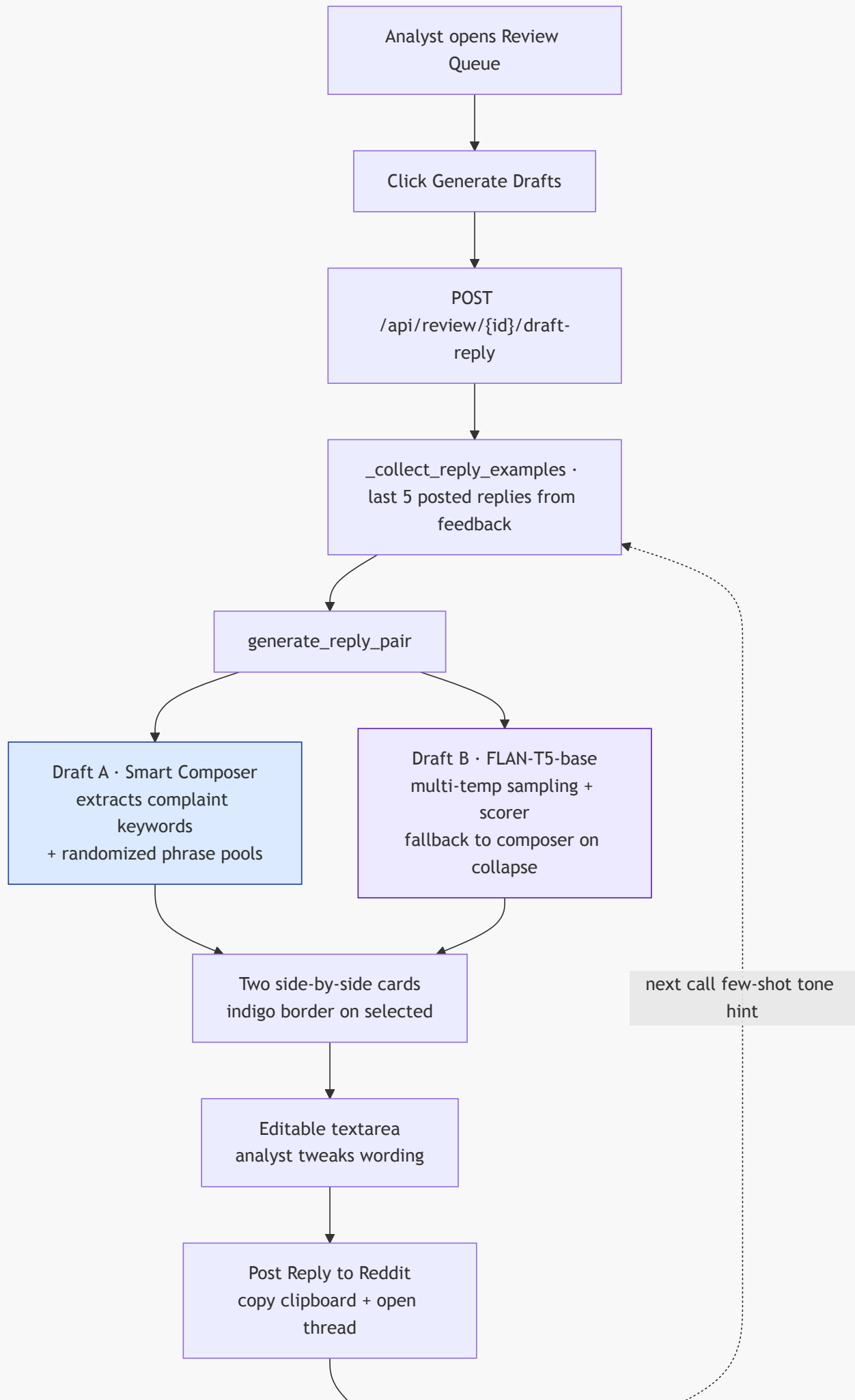


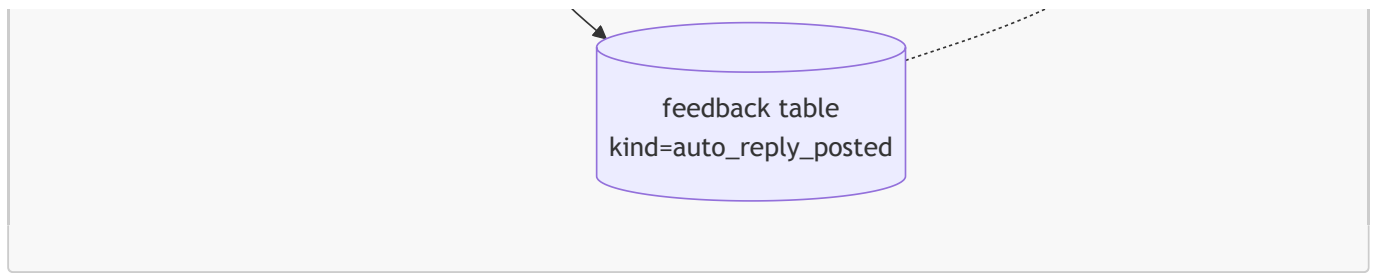
2. Pipeline stages (sequence)



3. Reply-generation learning loop





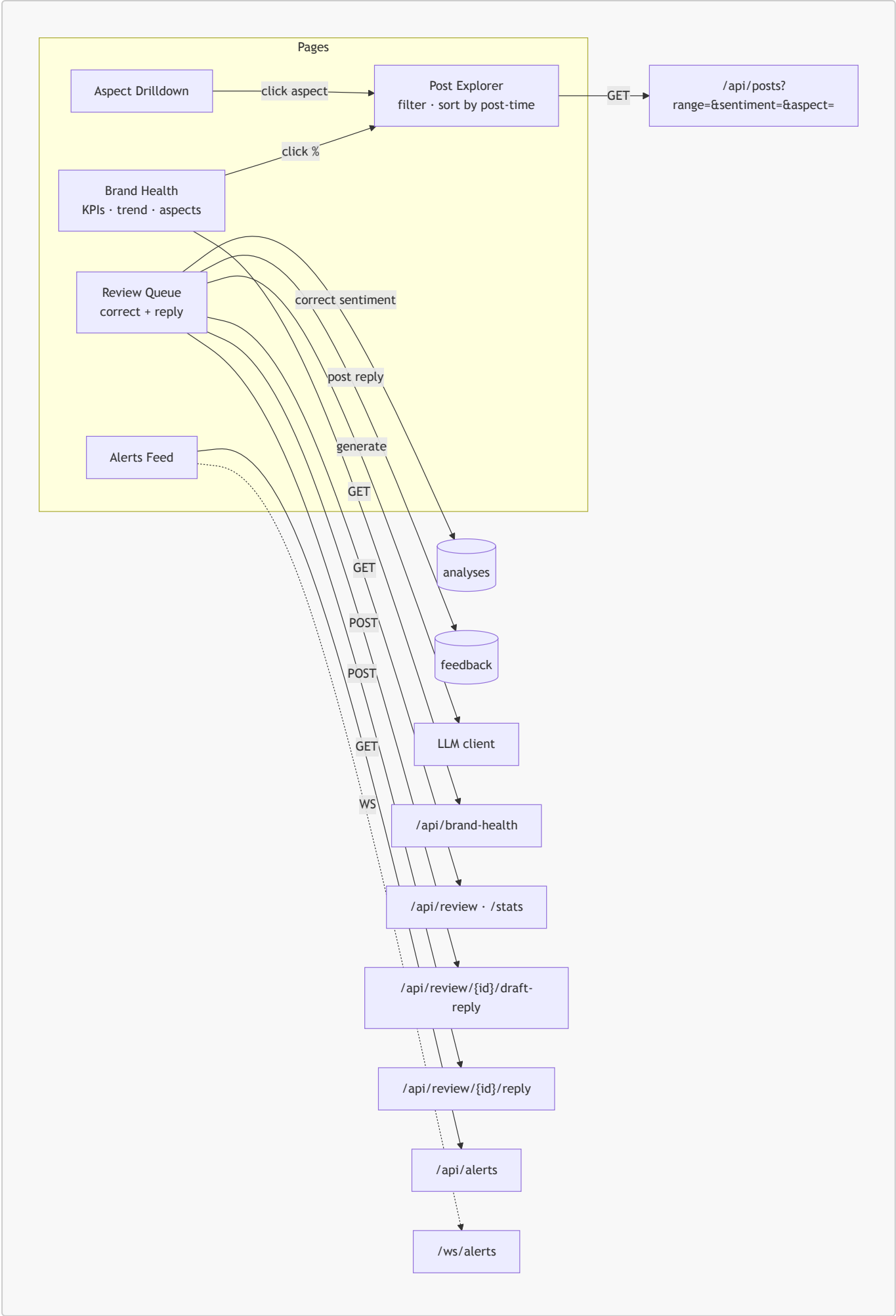


4. Tools & libraries by layer

Layer	Tool / Library	Purpose
Ingestion	<code>arctic-shift.photon-reddit.com</code> (HTTPS + <code>curl</code> subprocess)	Free, key-less Reddit data
	<code>praw</code> 7.7+	Optional official Reddit API path
	<code>langdetect</code>	English-only filter
	<code>sentence-transformers</code> (MiniLM-L6-v2)	Semantic dedup
Storage	<code>sqlite3</code> (built-in) → <code>data/local.db</code>	Local dev store
	<code>azure-cosmos</code>	Production store (configurable)
NLP / LLM	<code>transformers</code> + <code>torch</code>	Local model runtime (offline cache)
	<code>cardiffnlp/twitter-roberta-base-sentiment-latest</code>	Sentiment classification
	<code>facebook/bart-large-mnli</code>	Zero-shot aspect classification
	<code>google/flan-t5-base</code>	Reply text generation (Draft B)
	<code>openai</code> SDK	Azure OpenAI opt-in path
	Custom Smart Composer	Always-varied, content-aware reply (Draft A)
API	<code>fastapi</code> + <code>uvicorn</code> + <code>websockets</code>	REST + live alert push (port 8000)
	<code>pydantic</code>	Request / response schemas
Scheduling	<code>asyncio</code> event loop (lifespan)	60-min cron + manual Run Now
	<code>subprocess</code> (<code>python -m src.pipeline --once</code>)	Detached pipeline runs
Frontend	<code>React 18</code> + <code>TypeScript</code> + <code>Vite</code> (port 3001)	SPA
	<code>react-router-dom</code> v6	Routing (Brand Health · Posts · Review · Alerts · Aspects)
	<code>recharts</code>	Sentiment pie, trend lines

Layer	Tool / Library	Purpose
	<code>tailwindcss</code>	Styling
Observability	<code>structlog</code>	Structured JSON logs
	<code>data/llm_costs.jsonl</code> (<code>CostTracker</code>)	Per-call cost ledger
Config	<code>pyyaml</code> (<code>config/pipeline_config.yaml</code>) + <code>python-dotenv</code>	Pluggable provider switches
Testing	<code>pytest</code> , <code>pytest-asyncio</code> , <code>scikit-learn</code> (metrics)	Unit + integration + F1 / AUC eval

5. Request paths through the dashboard



6. TL;DR

- **No Reddit key** — Arctic Shift public archive over plain HTTPS via `curl`.
- **All LLMs run locally** — three HF models cached in `~/.cache/huggingface/`, started with `TRANSFORMERS_OFFLINE=1 HF_HUB_OFFLINE=1`.
- **One Python orchestrator** (`src/pipeline.py`) runs the 6-stage flow hourly and on-demand.
- **FastAPI** (`src/dashboard/api.py`) exposes the dashboard data + manual pipeline triggers; **React/Vite** renders 5 pages.
- **Closed feedback loop**: every analyst-posted reply lands in the `feedback` table and becomes a few-shot tone hint for the next draft.

7. Key files

Area	File
Pipeline orchestrator	src/pipeline.py
Arctic Shift fetcher	src/ingestion/arctic_shift.py
PRAW fetcher (optional)	src/ingestion/reddit_client.py , src/ingestion/fetcher.py
Preprocess + dedup	src/ingestion/preprocess.py , src/trust/dedup.py
Trust scoring	src/trust/scorer.py , src/trust/heuristics.py
Sentiment + aspects + reply	src/analysis/analyzer.py , src/analysis/llm_client.py
Aggregation + alerts	src/aggregation/aggregator.py , src/alerts/engine.py
Storage	src/storage/store.py , src/storage/cursor.py
API	src/dashboard/api.py
Config	config/pipeline_config.yaml
Frontend pages	frontend/src/pages/BrandHealth.tsx , frontend/src/pages/PostExplorer.tsx , frontend/src/pages/ReviewQueue.tsx , frontend/src/pages/AspectDrilldown.tsx , frontend/src/pages/AlertFeed.tsx